# Nesne Yönelimli Programlama

## Hafta 6: Yapılar, Birlikler ve Numaralandırmalar

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

# Hafta 6: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- Yapılar ( `struct` ) ve Bileşik Veri Modelleme:
  - Heterojen veri tiplerinin tek bir kullanıcı tanımlı tür altında toplanması
  - Üye erişim operatörleri ( `.` ve `->` ), yapı dizileri, iç içe yapılar ve kurucu/yıkıcılar
- Birlikler ( `union` ) ve Bellek Paylaşımı:
  - Çakışan ortak bellek başlangıcı, `sizeof` en büyük üye kuralı ve etiketli birlikler
- Endian Mimarisi ve Bayt Sıralaması:
  - Bellekte çok baytlı verilerin dizilimi (Little-Endian vs Big-Endian) ve tespiti
- Bellek Fragmantasyonu ve Hizalama:
  - Dahili parçalanma (*Internal Fragmentation*), dolgu baytları (*padding*) ve `#pragma pack`
- Bitwise İşlemler ve Numaralandırmalar:
  - Bit düzeyinde mantık/kaydırma, bit alanları (*bit-fields*) ve tür güvenli `enum class`

Yapı (struct): Farklı veri tiplerindeki ilişkili değişkenleri tek bir bellek bloğunda birleştiren kullanıcı tanımlı bileşik veri tipi.

# Hafta 6: Uygulamalı ve Mekatronik Hedefler

Laboratuvar oturumu ve gömülü sistem mimarisi kazanımları:

- Sensör ve Eyleyici Veri Paketleri:
  - Çok eksenli robot kol eklemlerinin ve IMU sensör verilerinin iç içe yapılarla modellenmesi
  - Büyük yapıların fonksiyonlara `const T&` ve `const T*` ile sıfır kopyalama maliyetiyle aktarılması
- Donanım Kaydedicisi (Register) ve Bitwise Maskeleme:
  - 8-bit ve 16-bit mikrodenetleyici kontrol/durum kaydedicilerinin bitwise ve `bit-fields` ile yönetimi
- Seri Haberleşme ve CAN Bus Protokol Paketleme:
  - `#pragma pack(push, 1)` ile dahili fragmantasyonun sıfırlanması ve 8-baytlık CAN çerçevesi tasarımı
- Sonlu Durum Makinesi (FSM) Denetimi:
  - `enum class : uint8_t` tabanlı tip güvenli endüstriyel robot durum makinesi implementasyonu

CAN Bus (Controller Area Network): Otomotiv ve endüstriyel otomasyonda mikrodenetleyicilerin gürültüye dayanıklı haberleşmesini sağlayan seri veri yolu standardı.

# Motivasyon: Dağınık Değişkenlerden Bileşik Veriye

Bir mekatronik sistemde motor veya sensör parametrelerini bağımsız değişkenlerle yönetmek ölçeklenemez:

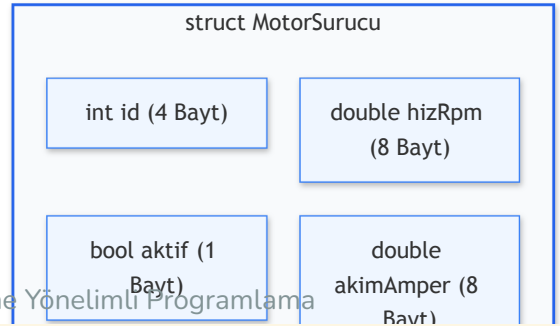
- **Veri Bütünlüğü Kaybı:** Bir motorun kimliği, hızı, akımı ve hata durumu ayrı değişkenlerde tutulduğunda aralarındaki mantıksal bağ yazılımda görünmez hale gelir.
- **Fonksiyon Parametre Karmaşası:** Bir fonksiyona 8 ayrı motor parametresi göndermek hem okunabilirliği bozar hem de çağrı yığını (*call stack*) yönetimini zorlaştırır.
- **Çoklu Nesne Zorluğu:** Sisteme 1 motor yerine 6 eksenli bir robot kol eklendiğinde, bağımsız değişkenleri kopyalamak ( `hiz1, hiz2, ...` ) hataya açık kod tekrarı üretir.
- **Mühendislik Çözümü:** İlgili tüm nitelikleri `struct` çatısı altında birleştirerek tek bir atomik birim olarak taşımak ve işlemek.

> **Mühendislik İlkesi:** Fiziksel dünyada aynı nesneye ait olan büyüklükler (sıcaklık, basınç, zaman damgası), yazılım mimarisinde de tek bir veri yapısı içinde kapsüllenmelidir.

# Yapı ( `struct` ) Tanımlama ve Sözdizimi

`struct` bildirimi derleyiciye yeni bir veri tipinin bellek şablonunu tanıtır:

- **Tip Şablonu (Blueprint):** `struct MotorSurucu { ... };` satırı tek başına bellekte yer kaplamaz; bir tip kalıbı oluşturur.
- **Fiziki Bellek Tahsisi:** Bu tipten bir değişken ( `MotorSurucu m1;` ) tanımlandığı anda bellekte yer ayrılır.
- **Noktalı Virgül Zorunluluğu:** Yapı gövdesini kapatan süslü parantezden sonra mutlaka noktalı virgül ( `;` ) konulmalıdır.
- **Heterojen Veri Havuzu:** Yapı içerisinde farklı türden temel değişkenler ( `int` , `double` , `bool` ) bir arada saklanır.



# Yapı Nesnesi Oluşturma ve Üye Erişimi ( `.` )

Yapı türünden nesne tanımlama ve **nokta operatörü** ( `.` ) ile üyelere erişim:

```
8   };
9
10  int main() {
11      MotorSurucu sagMotor; // Yığında (Stack) nesne oluşturulur
12      sagMotor.id = 1;
13      sagMotor.hizRpm = 1450.5;
14      sagMotor.akimAmper = 3.2;
15      sagMotor.aktif = true;
16
17      std::cout << "Motor ID   : " << sagMotor.id << "\n";
18      std::cout << "Hiz (RPM)  : " << sagMotor.hizRpm << "\n";
19      std::cout << "Akim (A)   : " << sagMotor.akimAmper << "\n";
20      std::cout << "Durum      : " << (sagMotor.aktif ? "CALISIYOR" : "DURDU") << "\n";
21      return 0;
22  }
```

```
1  Motor ID   : 1
2  Hiz (RPM)  : 1450.5
3  Akim (A)   : 3.2
4  Durum      : CALISIYOR
```

# Toplu İlklendirme (Aggregate Initialization)

Yapı üyeleri tanımlandıkları anda süslü parantez `{}` ile tek satırda ilklendirilebilir:

```
};

int main() {
    // 1. C-tarzı küme parantezi ile ilklendirme (Bildirim sırasına göre)
    SensorVerisi s1 = {1, 3.32, 42.5};

    // 2. Modern C++11 tekbiçimli (uniform) ilklendirme
    SensorVerisi s2{2, 4.95, 28.1};

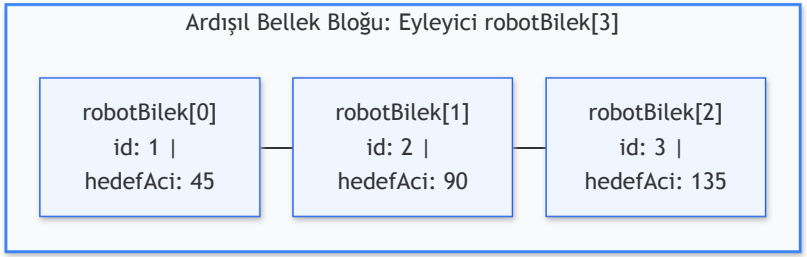
    std::cout << "Sensor " << s1.kanalNo << " -> Gerilim: " << s1.gerilim
              << " V | Sicaklik: " << s1.sicaklik << " C\n";
    std::cout << "Sensor " << s2.kanalNo << " -> Gerilim: " << s2.gerilim
              << " V | Sicaklik: " << s2.sicaklik << " C\n";
    return 0;
}
```

```
Sensor 1 -> Gerilim: 3.32 V | Sicaklik: 42.5 C
Sensor 2 -> Gerilim: 4.95 V | Sicaklik: 28.1 C
```

# Yapı Dizileri (Array of Structs)

Çok kanallı sensör veya çok eksenli robot sistemlerinde aynı yapı tipinden oluşan diziler kullanılır:

- **Ardışıl Bellek Yerleşimi:** Yapı dizisinin her bir elemanı bellekte kesintisiz ve yan yana sıralanır ( `dizi[0]` , `dizi[1]` , `dizi[2]` ).
- **Erişim Sözdizimi:** Önce dizi indisi `[]` ile ilgili nesne seçilir, ardından nokta `.` operatörü ile üyesine ulaşılır: `robotBilek[i].hedefAci` .
- **Döngü Entegrasyonu:** Aralık tabanlı `for` döngüsünde `const T&` kullanılarak tüm yapı elemanları kopyalanmadan hızla taranır.



# Yapı Dizileri: Çoklu Eyleyici Kontrolü

3 serbestlik dereceli bir robot bileğinin eklem verilerini yapı dizisi ile yöneten uygulama:

```
    double torkNm;   // Newton-metre
};

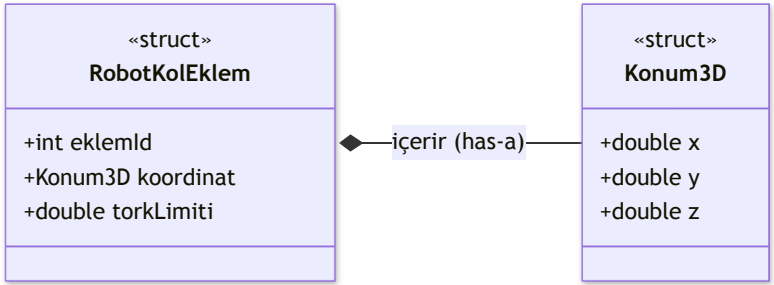
int main() {
    const int EKSEN_SAYISI = 3;
    Eyleyici robotBilek[EKSEN_SAYISI] = {
        {1, 45, 1.2}, {2, 90, 2.5}, {3, 135, 0.8}
    };

    for (const Eyleyici& motor : robotBilek) {
        std::cout << "Eksen " << motor.id << " -> Aci: " << motor.hedefAci
                  << " deg | Tork: " << motor.torkNm << " Nm\n";
    }
    return 0;
}
```

```
Eksen 1 -> Aci: 45 deg | Tork: 1.2 Nm
Eksen 2 -> Aci: 90 deg | Tork: 2.5 Nm
Eksen 3 -> Aci: 135 deg | Tork: 0.8 Nm
```

# İç İçe Yapılar (Nested Structs) ve Kompozisyon

Karmaşık mekatronik sistemler, daha küçük alt yapıların birleştirilmesiyle (*kompozisyon*) modellenir:




- **Zincirleme Erişim:** İç yapı üyelerine kademeli nokta operatörüyle erişilir: `eklem1.koordinat.z`.
- **Yeniden Kullanılabilirlik:** `Konum3D` yapısı hem robot ekleminde hem de lidar hedef tespitinde ortak kullanılır.

# İç İçe Yapılar: Robot Kol Eklem İmplementasyonu

```
struct RobotKolEklem {
    int eklemId;
    Konum3D koordinat; // İç içe yapı üyesi
    double torkLimiti;
};

int main() {
    RobotKolEklem eklem1 = {101, {120.5, -45.0, 310.2}, 65.0};

    std::cout << "Eklem ID        : " << eklem1.eklemId << "\n";
    std::cout << "Konum (X, Y, Z) : (" << eklem1.koordinat.x << ", "
              << eklem1.koordinat.y << ", " << eklem1.koordinat.z << ") mm\n";
    std::cout << "Tork Limiti     : " << eklem1.torkLimiti << " Nm\n";
    return 0;
}
```

```
Eklem ID        : 101
Konum (X, Y, Z) : (120.5, -45, 310.2) mm
Tork Limiti     : 65 Nm
```

# Yapı İşaretçileri ve Ok ( `->` ) Operatörü

Bir yapı nesnesine işaretçi (*pointer*) üzerinden erişirken **ok operatörü** ( `->` ) kullanılır:

- **Dolaylı Üye Erişimi:** `ptr->uye` ifadesi, işaretçinin gösterdiği nesneye ulaşıp üyesini okuyan `(*ptr).uye` ifadesinin birebir karşılığıdır.
- **Operatör Önceliği Riski:** Nokta ( `.` ) operatörü, dereferans ( `*` ) operatöründen daha yüksek önceliğe sahiptir; bu nedenle `*ptr.uye` yazımı derleme hatası üretir.
- **Bellek Verimliliği:** Büyük yapı nesneleri fonksiyonlara kopyalanmak yerine adresleri ( `const Yapı*` ) veya referansları ( `const Yapı&` ) üzerinden iletilir.

> **Sözdizimi Eşdeğerliği:**
> • Doğrudan nesne üzerinden: `paket.bataryaGerilimi`
> • İşaretçi üzerinden (Tercih edilen): `ptr->bataryaGerilimi`
> • Parantezli dereferans ile: `(*ptr).bataryaGerilimi`

# Yapıların Fonksiyonlara Aktarımı (`const T*` ve `const T&`)

Büyük yapıları kopyalamadan verimli ve güvenli şekilde fonksiyonlara aktarma:

```
12      std::cout << "[PTR] Zaman: " << ptr->zamanDamgasiMs << " ms | Batarya: " << ptr->batar
13  }
14
15  // Referans ile aktarım: Modern C++ için tercih edilen sıfır kopyalı yöntem
16  void telemetriYazdirRef(const TelemetriPaketi& ref) {
17      std::cout << "[REF] Zaman: " << ref.zamanDamgasiMs << " ms | Batarya: " << ref.batarya
18  }
19
20  int main() {
21      TelemetriPaketi paket = {124500, 24.6f, 0};
22
23      telemetriYazdirPtr(&paket); // Adres aktarılır
24      telemetriYazdirRef(paket);  // Takma ad (referans) aktarılır (Kopyalama yok!)
25      return 0;
26  }
```

```
1  [PTR] Zaman: 124500 ms | Batarya: 24.6 V
2  [REF] Zaman: 124500 ms | Batarya: 24.6 V
```

# Dinamik Bellekte Yapı Tahsisi ( `new` / `delete` )

Çalışma zamanında (*run-time*) oluşturulan yapı nesneleri Öbek (*Heap*) bellekte tahsis edilir:

```
4       long pulsSayaci;
5       int donusHiziRpm;
6   };
7
8   int main() {
9       // Heap üzerinde dinamik yapı tahsisi ve ilklendirme
10      EnkoderVerisi* enkPtr = new EnkoderVerisi{1048576L, 3000};
11
12      std::cout << "Enkoder Puls: " << enkPtr->pulsSayaci
13                << " | Hiz: " << enkPtr->donusHiziRpm << " RPM\n";
14
15      delete enkPtr;    // Tahsis edilen öbek belleği sisteme iade edilir
16      enkPtr = nullptr; // Sarkan gösterici (dangling pointer) önlenir
17      return 0;
18  }
```

```
1   Enkoder Puls: 1048576 | Hiz: 3000 RPM
```

## `struct` Yapısı

Varsayılan erişim ve kalıtım `public` 'tir:

```
1  struct YapiNokta {
2      int x; // Varsayılan: public
3      int y;
4  };
5
6  int main() {
7      YapiNokta p;
8      p.x = 10; // Doğrudan erişilebilir
9  }
```

- Kullanım Amacı: Pasif veri paketleri, donanım mesaj çerçeveleri ve koordinat demetleri.

## `class` Yapısı

Varsayılan erişim ve kalıtım `private` 'tır:

```
1  class SinifNokta {
2      int x; // Varsayılan: private
3      int y;
4  public:
5      void ayarla(int a, int b) {
6          x = a; y = b;
7      }
8  };
```

- Kullanım Amacı: Veri gizleme (*data hiding*) ve sınıf değişmezlerini (*invariants*) koruyan nesneler.

# Yapılarda Kurucu Metot (Constructor)

C++ dilinde yapılar, nesne oluşturulduğu anda otomatik çalışan **kurucu metotlara** sahip olabilir:

```
    int pwmPin;
    double maksAkim;

    // Üye ilklendirme listesi (Member Initializer List) ile kurucu
    DCMotor(int _id, int _pin, double _akim)
        : id(_id), pwmPin(_pin), maksAkim(_akim) {
        std::cout << "Motor " << id << " (Pin " << pwmPin << ") basariyla kuruldu.\n";
    }
};

int main() {
    DCMotor sagMotor(1, 9, 4.5); // Kurucu otomatik çağrılır
    std::cout << "Maksimum Akim: " << sagMotor.maksAkim << " A\n";
    return 0;
}
```

```
Motor 1 (Pin 9) basariyla kuruldu.
Maksimum Akim: 4.5 A
```

# Aşırı Yüklenmiş Kurucular (Constructor Overloading)

Bir yapı, farklı başlatma senaryolarını desteklemek üzere birden fazla kurucu barındırabilir:

```
    // 1. Varsayılan (Default) Kurucu
    KalmanDurumu() : tahmin(0.0), hataKovaryansi(1.0) {}

    // 2. Parametreli Kurucu
    KalmanDurumu(double t, double p) : tahmin(t), hataKovaryansi(p) {}
};

int main() {
    KalmanDurumu k1;             // Varsayılan kurucu: (0.0, 1.0)
    KalmanDurumu k2(24.5, 0.05); // Parametreli kurucu: (24.5, 0.05)

    std::cout << "K1: Tahmin=" << k1.tahmin << " | Hata=" << k1.hataKovaryansi << "\n";
    std::cout << "K2: Tahmin=" << k2.tahmin << " | Hata=" << k2.hataKovaryansi << "\n";
    return 0;
}
```

```
K1: Tahmin=0 | Hata=1
K2: Tahmin=24.5 | Hata=0.05
```

# Yapılarda Yıkıcı Metot (Destructor) ve RAII

Yapı nesnesi kapsam dışına çıktığında (*out of scope*) yıkıcı metot ( `~YapiAdi` ) otomatik çalışır:

```
11
12      ~SensorTamponu() {
13          delete[] veriAlani; // Kapsam bitiminde otomatik bellek iadesi
14          std::cout << "[YIKICI] Tampon bellegi guvenle serbest birakildi.\n";
15      }
16  };
17
18  int main() {
19      std::cout << "Ana program basladi.\n";
20      {
21          SensorTamponu localTampon(10); // Kapsam içi nesne oluşturuldu
22      } // Kapsam kapandı -> Yıkıcı otomatik olarak burada tetiklenir!
23      std::cout << "Ana program sona erdi.\n";
24      return 0;
25  }
```

```
1  Ana program basladi.
2  [KURUCU] 10 elemanli dinamik tampon ayrildi.
3  [YIKICI] Tampon bellegi guvenle serbest birakildi.
4  Ana program sona erdi.
```

# Birlikler ( `union` ) ve Ortak Bellek Paylaşımı

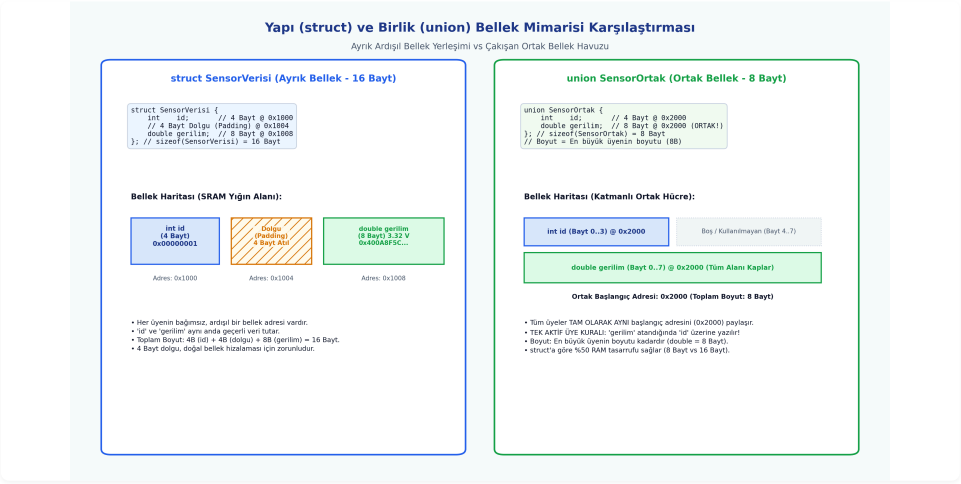
Birlik ( `union` ), tüm üyelerinin **aynı bellek başlangıç adresini** paylaştığı özel bir veri yapısıdır:

- **Ortak Bellek Havuzu:** Yapılarda ( `struct` ) her üye için ayrı bellek alanı tahsis edilirken, birliklerde tüm üyeler aynı fiziki bellek hücresinin üzerine yerleşir.
- **Boyut Kuralı:** Bir birliğin toplam bellek boyutu ( `sizeof` ), içindeki **en büyük üyenin boyutuna** eşittir.
- **Tek Aktif Üye İlkesi:** Herhangi bir anda birlikteki üyelerden **yalnızca biri** anlamlı veri tutabilir; yeni bir üyeye atama yapıldığında önceki üyenin bit dizilimi değişir.
- **Gömülü Sistem Kullanımı:** Kısıtlı RAM'e sahip mikrodenetleyicilerde ortak tampon paylaşımı ve donanım kaydedici erişimi için kullanılır.

> **Birlik (union):** Üyelerinin tamamı aynı bellek adresinden başlayan ve aynı anda yalnızca tek bir üyesi aktif olan düşük seviyeli veri yapısı.

# `struct` ve `union` Bellek Mimarisi Karşılaştırması

Ayrık ardışıl bellek yerleşimi ( `struct` ) ile çakışan ortak bellek yerleşiminin ( `union` ) karşılaştırması:




- `struct SensorVerisi` : `id` ( `0x1000` ) ve `gerilim` ( `0x1008` ) ayrı adreslerdedir; aradaki 4B dolgu ile toplam boyut **16 Bayt**'tır.
- `union SensorOrtak` : Hem `id` hem de `gerilim` `0x2000` adresinden başlar; toplam boyut **8 Bayt**'tır.

# Birliklerde Tek Aktif Üye Davranışı

Birlik içinde bir üyeye değer yazıldığında ortak bellek alanı güncellenir:

```
union DonanimKaydi {
    int   tamSayi;   // 4 Bayt @ Adres 0x00
    float gerilim;   // 4 Bayt @ Adres 0x00 (Ortak!)
};

int main() {
    DonanimKaydi reg;
    reg.tamSayi = 100;
    std::cout << "Tam sayi aktif : " << reg.tamSayi << "\n"; // 100 (Gecerli)

    reg.gerilim = 3.3f; // Ortak 4 baytlik alanin uzerine IEEE 754 float yazildi!
    std::cout << "Gerilim aktif  : " << reg.gerilim << " V\n"; // 3.3 (Gecerli)
    std::cout << "Eski tam sayi  : " << reg.tamSayi << "\n";   // 1079194419 (Bozuldu!)
    return 0;
}
```

```
Tam sayi aktif : 100
Gerilim aktif  : 3.3 V
Eski tam sayi  : 1079194419
```

Açıklama: `1079194419` sayısı, `3.3f` ondalıklı sayısının IEEE-754 ikili karşılığı olan `0x40533333` bit dizilimının tamsayı yorumudur.

# Endian Mimarisi: Little-Endian vs Big-Endian

Çok baytlı verilerin ( `int` , `float` ) bellekte hangi bayt sırasıyla saklanacağını belirleyen mimari kural:

- **Little-Endian (En Küçük Bayt Önce):**
  - En düşük anlamlı bayt (*LSB - Least Significant Byte*), en küçük bellek adresine yazılır.
  - x86/x64 bilgisayarlar ve ARM Cortex-M mikrodenetleyicileri standart olarak **Little-Endian** çalışır.
- **Big-Endian (En Büyük Bayt Önce):**
  - En yüksek anlamlı bayt (*MSB - Most Significant Byte*), en küçük bellek adresine yazılır.
  - Ağ protokolleri (TCP/IP - *Network Byte Order*) ve bazı endüstriyel veri yolları Big-Endian kullanır.
- **Mekatronik Önemi:** Mikrodenetleyiciden çıkan Little-Endian bir veri, ağ üzerinden gönderilirken bayt sırası ters çevrilmelidir!

# `union` ile Çalışma Zamanında Endianlık Tespiti

Bir işlemcinin Little-Endian mı yoksa Big-Endian mı çalıştığını test eden C++ programı:

```cpp
    uint16_t deger;      // 2 Bayt: 0x0102 (MSB: 0x01, LSB: 0x02)
    uint8_t  baytlar[2]; // 2 x 1 Bayt
};

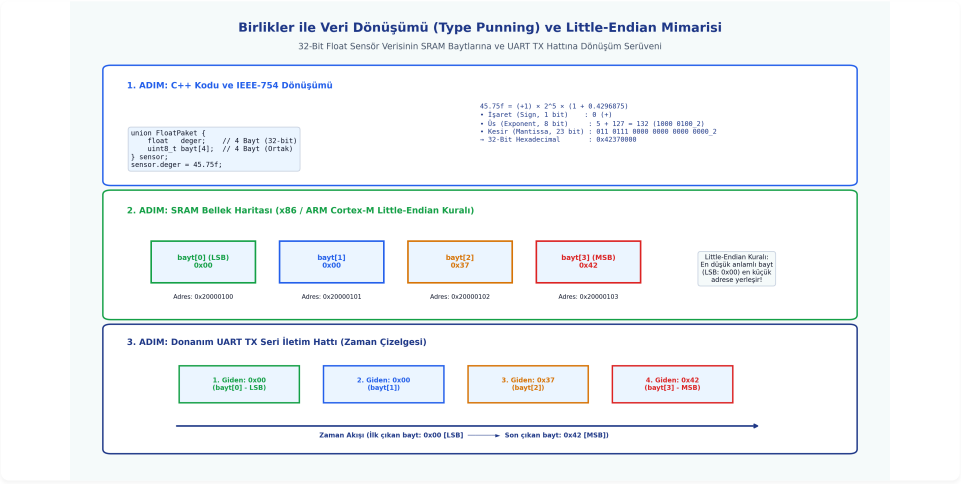
int main() {
    EndianTest test;
    test.deger = 0x0102;

    if (test.baytlar[0] == 0x02) {
        std::cout << "Mimari: LITTLE-ENDIAN (LSB en kucuk adreste: 0x02)\n";
    } else {
        std::cout << "Mimari: BIG-ENDIAN (MSB en kucuk adreste: 0x01)\n";
    }
    return 0;
}
```

```
Mimari: LITTLE-ENDIAN (LSB en kucuk adreste: 0x02)
```

# Birlikler ile Bayt Düzeyinde Ayrıştırma (Type Punning)

Gömülü haberleşme protokollerinde (UART, SPI, I2C, CAN) çok baytlı veriler bayt dizisine dönüştürülür:




- **Sıfır Hesaplama Yükü:** 32-bit `float` sensör verisi, `uint8_t bayt[4]` dizisi ile aynı 4 baytlık belleği paylaşır.
- **Little-Endian Sıralaması:** `45.75f` ( `0x42370000` ) değeri belleğe `0x00` , `0x00` , `0x37` , `0x42` olarak yerleşir ve UART hattına bu sırayla basılır.

# 32-Bit Sensör Verisinin Baytlara Ayrıştırılması

```
    uint8_t bayt[4];   // 4 x 1 Bayt (işaretsiz 8-bit dizi)
};

int main() {
    FloatPaket sensor;
    sensor.deger = 45.75f; // IEEE 754: 0x42370000

    std::cout << "Float Deger: " << sensor.deger << "\nUART TX Baytlari: ";
    for (int i = 0; i < 4; ++i) {
        std::cout << "0x" << std::hex << std::uppercase << std::setw(2)
                  << std::setfill('0') << static_cast<int>(sensor.bayt[i]) << " ";
    }
    std::cout << std::dec << "\n";
    return 0;
}
```

```
Float Deger: 45.75
UART TX Baytlari: 0x00 0x00 0x37 0x42
```

# Etiketli Birlik Tasarımı (Tagged Union)

Bir birliğin o anda hangi üyesinin geçerli olduğunu güvenle takip etmek için **Etiketli Birlik** deseni kullanılır:

- **Problem:** Çıplak bir `union` kendi içinde hangi üyenin aktif olduğu bilgisini taşımaz; yanlış üyenin okunması veri hatasına yol açar.
- **Mimari Çözüm:** Birlik yapısı, aktif veri tipini belirten bir `enum class` etiketi (*tag*) ile birlikte kapsayıcı bir `struct` içine yerleştirilir.
- **Anonim Birlik (Anonymous Union):** Yapı içinde isimsiz tanımlanan birlik üyelerine doğrudan dış yapı üzerinden erişilebilir.
- **Modern C++ Karşılığı:** C++17 standardındaki `std::variant` sınıfı, etiketli birlik mimarisinin tür güvenli şablon karşılığıdır.

> **Etiketli Birlik (Tagged / Discriminated Union):** Ortak bellek alanındaki aktif veri tipini bir durum etiketi ( `enum` ) ile birlikte saklayan güvenli yapı deseni.

# Etiketli Birlik C++ İmplementasyonu

```cpp
    }
}

int main() {
    GuvenliTelemetri t1;
    t1.aktifTip = VeriTipi::SICAKLIK;
    t1.sicaklik = 36.8f;
    paketiIsle(t1);

    GuvenliTelemetri t2;
    t2.aktifTip = VeriTipi::MOTOR_HIZI;
    t2.hizRpm = 2800;
    paketiIsle(t2);
    return 0;
}
```

```
[TELEMETRI] Sicaklik : 36.8 C
[TELEMETRI] Motor Hiz: 2800 RPM
```

# Bit Düzeyinde Mantık Operatörleri (Bitwise Operators)

Gömülü sistemlerde donanım pinleri ve kaydediciler bit düzeyinde denetlenir:

| Operatör | Anlamı | İşlem Kuralı | Örnek ($A = 1010_2, B = 1100_2$) |
|---|---|---|---|
| `&` | VE (AND) | Her iki bit de 1 ise sonuç 1 | $A \,\&\, B = 1000_2$ |
| `\|` | VEYA (OR) | En az bir bit 1 ise sonuç 1 | $A \mid B = 1110_2$ |
| `^` | ÖZEL VEYA (XOR) | Bitler farklıysa 1, aynıysa 0 | $A \hat{} B = 0110_2$ |
| `~` | DEĞİL (NOT) | Tüm bitleri tersler ($0 \to 1, 1 \to 0$) | $\sim A = 0101_2$ |
| `<<` | Sola Kaydırma | Bitleri sola kaydırır ($2^k$ ile çarpma) | $A \ll 1 = 0100_2$ (Taşma hariç) |
| `>>` | Sağa Kaydırma | Bitleri sağa kaydırır ($2^k$'ye bölme) | $A \gg 1 = 0101_2$ |

# Donanım Kaydedicilerinde Bit Maskeleme Teknikleri

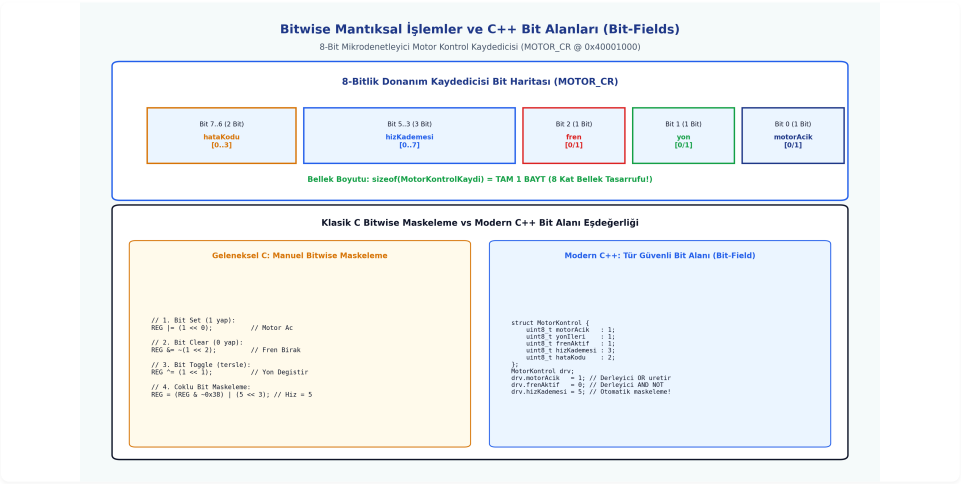
Mikrodenetleyici kaydedicilerinde 4 temel bit operasyonu:

```
9       std::cout << "Bit 0 Set Sonrasi  : 0x" << std::hex << (int)REG << "\n";
10
11      // 2. BIT CLEAR (0 Yapma): AND NOT (& ~) operatörü ile
12      REG &= ~(1 << 0); // Bit 0'ı 0 yap (Motoru durdur)
13      std::cout << "Bit 0 Clear Sonrasi: 0x" << std::hex << (int)REG << "\n";
14
15      // 3. BIT TOGGLE (Tersleme): XOR (^) operatörü ile
16      REG ^= (1 << 1); // Bit 1'i tersle (0 ise 1, 1 ise 0 yap)
17      std::cout << "Bit 1 Toggle       : 0x" << std::hex << (int)REG << "\n";
18
19      // 4. BIT CHECK (Durum Sorgulama): AND (&) ile
20      bool yonIleri = (REG & (1 << 1)) != 0;
21      std::cout << "Yon Ileri mi?      : " << (yonIleri ? "EVET" : "HAYIR") << "\n";
22      return 0;
23  }
```

```
1  Bit 0 Set Sonrasi  : 0x1
2  Bit 0 Clear Sonrasi: 0x0
3  Bit 1 Toggle       : 0x2
4  Yon Ileri mi?      : EVET
```

# Bit Alanları (Bit-Fields) ve Donanım Modelleme

Manuel bitwise maskeleme yerine C++ dilinin sunduğu tür güvenli **Bit Alanları** mimarisi:




- **Bit Genişliği Bildirimi:** `uint8_t uye : bitSayisi;` sözdizimi ile üyeye sadece gereken bit adedi tahsis edilir.
- **Derleyici Dönüşümü:** `drv.hizKademesi = 5;` yazıldığında derleyici arka planda aynı `(REG & ~mask) | (val << shift)` makine kodunu üretir!

# Bit Alanları ile 8-Bit Motor Kontrol Kaydedicisi

```
 8       uint8_t hizKademesi : 3; // Bit 3-5 (3 bit -> 0..7 arasi 8 kademe)
 9       uint8_t hataKodu    : 2; // Bit 6-7 (2 bit -> 0..3 arasi 4 durum)
10   };
11
12   int main() {
13       MotorKontrolKaydi drv = {1, 1, 0, 5, 0};
14
15       std::cout << "Kaydedici Boyutu : " << sizeof(MotorKontrolKaydi) << " Bayt\n";
16       std::cout << "Motor Yonu       : " << (drv.yonIleri ? "Ileri" : "Geri") << "\n";
17       std::cout << "Hiz Kademesi     : " << static_cast<int>(drv.hizKademesi) << " / 7\n";
18
19       drv.frenAktif = 1; // Tek satirda freni devreye al
20       std::cout << "Fren Durumu      : " << (drv.frenAktif ? "KILITLI" : "SERBEST") << "\n";
21       return 0;
22   }
```

```
1   Kaydedici Boyutu : 1 Bayt
2   Motor Yonu       : Ileri
3   Hiz Kademesi     : 5 / 7
4   Fren Durumu      : KILITLI
```

# Hibrit Kaydedici: Birlik ve Bit Alanı Sentezi

Donanım sürücülerinde kaydediciye hem **tekil bitler** halinde hem de **tek bir ham bayt** olarak erişilir:

```
12      } bitler;
13  };
14
15  int main() {
16      DurumKaydedici reg;
17      reg.hamBayt = 0x05; // 0000 0101_2 -> hazir = 1, asiriAkim = 1
18
19      std::cout << "Ham Kaydedici: 0x" << std::hex << (int)reg.hamBayt << std::dec << "\n";
20      if (reg.bitler.asiriAkim) {
21          std::cout << "[ALARM] Asiri akim algilandi! Kaydedici tek komutta sifirlaniyor.\n";
22          reg.hamBayt = 0x00; // Tüm bitleri tek satırda sıfırla
23      }
24      std::cout << "Sifirlama Sonrasi Hazir mi: " << (int)reg.bitler.hazir << "\n";
25      return 0;
26  }
```

```
1  Ham Kaydedici: 0x5
2  [ALARM] Asiri akim algilandi! Kaydedici tek komutta sifirlaniyor.
3  Sifirlama Sonrasi Hazir mi: 0
```

# Bit Alanlarında Kısıtlamalar ve Mimari Kurallar

Bit alanları donanım seviyesinde büyük kolaylık sağlasa da C++ standardında önemli kısıtlara sahiptir:

- **Adres Operatörü ( `&` ) Yasaktır:** Bellek adreslemesi bayt sınırlarında yapılır; tekil bir bitin adresi alınamaz ( `&reg.bitler.hazir` derleme hatası verir).
- **İşaretçi ve Referans Bağlanamaz:** Bit alanlarına `uint8_t*` göstericisi veya `uint8_t&` referansı atanamaz.
- **Bit Dizilim Yönü (Implementation-Defined):** Bitlerin düşük bitten (*LSB*) mi yoksa yüksek bitten (*MSB*) mi yerleşeceği hedef işlemci mimarisine bağlıdır.
- **Değer Taşması (Bit Truncation):** $k$ bitlik bir işaretsiz alana $[0, 2^k - 1]$ aralığı dışında değer atanırsa üst bitler sessizce kesilir.

> **Tasarım Notu:** Bit alanlarında işaret bitinden kaynaklanan sürprizleri önlemek için daima açıkça işaretsiz tamsayı tipleri ( `uint8_t` , `uint16_t` , `uint32_t` ) tercih edilmelidir.

# Bellek Hizalama (Memory Alignment) ve CPU Veri Yolu

Modern 32-bit ve 64-bit işlemciler belleği tek tek baytlar halinde değil, 4 veya 8 baytlık kelimeler (*words*) halinde okur:

- **Doğal Hizalama Kuralı:** $N$ bayt boyutundaki bir temel veri tipinin bellek başlangıç adresi, $N$ sayısının tam katı olmalıdır:
  - `char` / `uint8_t` (1 Bayt) $\rightarrow$ Herhangi bir adrese yerleşebilir.
  - `int16_t` (2 Bayt) $\rightarrow$ Adresi 2'nin katı olmalıdır ( `0x1000` , `0x1002` , ...).
  - `int32_t` / `float` (4 Bayt) $\rightarrow$ Adresi 4'ün katı olmalıdır ( `0x1000` , `0x1004` , ...).
  - `double` (8 Bayt) $\rightarrow$ Adresi 8'in katı olmalıdır ( `0x1000` , `0x1008` , ...).
- **İşlemci Veri Yolu Kazancı:** Doğal hizalanmış veri tek bir bellek çevriminde (1 Bus Cycle) okunur; hizalanmamış veri ise 2 çevrim ve birleştirme gecikmesi üretir.

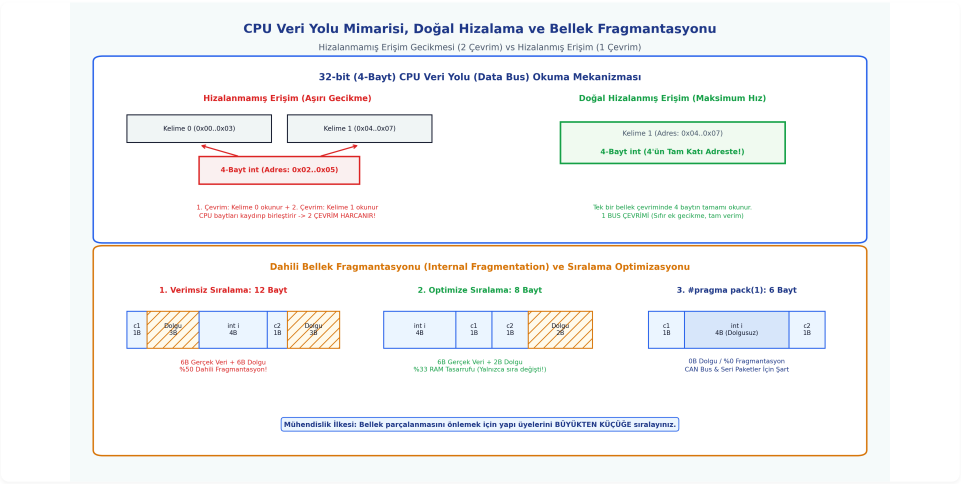
# Bellek Fragmantasyonu (Parçalanma) Kavramı

Gömülü sistemlerde bellek parçalanması sistem kararlılığını doğrudan etkiler:

- Dahili Parçalanma (Internal Fragmentation):
  - Yapı elemanlarını doğal sınırlara oturtmak için derleyicinin eklediği **dolgu baytları** (*padding*) nedeniyle oluşan atıl alanlardır.
  - Örnek: 6 baytlık verinin 12 bayt yer kaplaması $\longrightarrow$ 6 Bayt (%50) dahili parçalanma!
- Harici Parçalanma (External Fragmentation):
  - Öbek (*Heap*) üzerinde dinamik nesnelerin ( `new` / `delete` ) düzensiz oluşturulup silinmesiyle serbest bellek bloklarının delik deşik olması durumudur.
- **Mekatronik Riski:** Kısıtlı RAM'e (örneğin 2 KB - 64 KB) sahip mikrodenetleyicilerde aşırı parçalanma sistem kilitlenmelerine yol açar.

# CPU Veri Yolu, Hizalama ve Fragmantasyon Analizi

İşlemcinin veri okuma performansı ve yapı sıralamasının dahili parçalanmaya etkisi:




- **Sıralama Optimizasyonu:** Üyeleri büyükten küçüğe ( `int` , `char` , `char` ) dizmek dahili parçalanmayı %50'den %25'e düşürür.
- `#pragma pack(1)` : Tüm dolguyu sıfırlar (%0 parçalanma); ancak işlemcinin bellek erişim çevrimini artırabilir.

# Verimsiz vs Optimize Sıralama Karşılaştırması

```
    char c2; // 1B + 3B Son Dolgu
}; // sizeof = 12 Bayt (6B Veri + 6B Dahili Fragmantasyon!)

struct Verimli {
    int  i;  // 4B
    char c1; // 1B
    char c2; // 1B + 2B Son Dolgu
}; // sizeof = 8 Bayt (6B Veri + 2B Dolgu -> %33 Tasarruf!)

int main() {
    std::cout << "Verimsiz Boyut : " << sizeof(Verimsiz) << " Bayt\n";
    std::cout << "Verimli Boyut  : " << sizeof(Verimli) << " Bayt\n";
    std::cout << "Kazanc         : " << sizeof(Verimsiz) - sizeof(Verimli) << " Bayt Tasarr
    return 0;
}
```

```
Verimsiz Boyut : 12 Bayt
Verimli Boyut  : 8 Bayt
Kazanc         : 4 Bayt Tasarruf!
```

# `#pragma pack` ve C++11 `alignof` / `alignas`

Haberleşme çerçevelerinde dolguyu kapatma ve donanım DMA tamponlarında özel hizalama:

```
    uint8_t  bastik;   // 1 Bayt
    uint32_t sensorId; // 4 Bayt (Dolgusuz hemen 1. bayttan başlar)
    uint8_t  crc;      // 1 Bayt
}; // sizeof(SeriPaket) = 6 Bayt (%0 Dahili Fragmantasyon)
#pragma pack(pop)     // Varsayılan derleyici hizalamasına geri dön

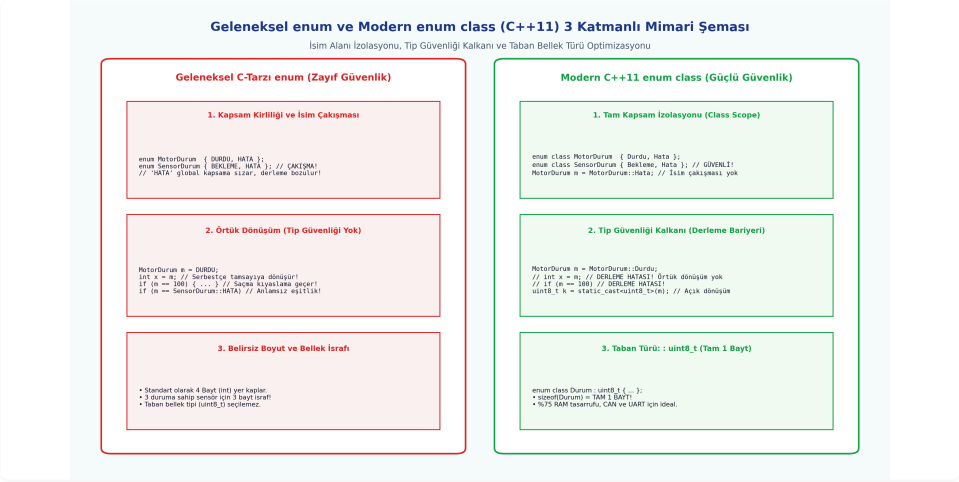
struct alignas(16) DmaTamponu {
    float kanallar[4]; // 16 Bayt
};

int main() {
    std::cout << "SeriPaket Boyutu   : " << sizeof(SeriPaket) << " Bayt\n";
    std::cout << "DmaTamponu Hizalama: " << alignof(DmaTamponu) << " Bayt\n";
    return 0;
}
```

```
SeriPaket Boyutu   : 6 Bayt
DmaTamponu Hizalama: 16 Bayt
```

# Geleneksel `enum` vs Modern `enum class` (C++11)

Klasik C numaralandırmalarının kapsam ve tip zafiyetleri ile modern `enum class` çözümü:




- **Kapsam Güvenliği:** `enum class` sabitlerine yalnızca `TipAdi::Sabit` ( `MotorDurum::Hata` ) şeklinde erişilir; global isim alanı kirlenmez.
- **Örtük Dönüşüm Engeli:** `enum class` değerleri tamsayılara kendiliğinden dönüşmez; farklı iki enum tipi birbirine karıştırılamaz.

# `enum class` ve Taban Bellek Türü ( `: uint8_t` )

Gömülü sistem protokollerinde numaralandırmanın bellekte kaplayacağı bayt boyutu açıkça belirlenir:

```
4  // Taban türü 1 bayt (uint8_t) olarak sabitlenmiş kapsamlı numaralandırma
5  enum class HataKodu : uint8_t {
6      HataYok       = 0x00,
7      AsiriGerilim  = 0x01,
8      AsiriSicaklik = 0x02,
9      IletisimKopuk = 0x03
10 };
11
12 int main() {
13     HataKodu durum = HataKodu::AsiriSicaklik;
14
15     std::cout << "sizeof(HataKodu): " << sizeof(HataKodu) << " Bayt\n";
16     std::cout << "Sayisal Kod     : " << static_cast<int>(durum) << "\n";
17     return 0;
18 }
```

```
1  sizeof(HataKodu): 1 Bayt
2  Sayisal Kod     : 2
```

# Tür Takma Adları: `typedef` vs Modern `using` (C++11)

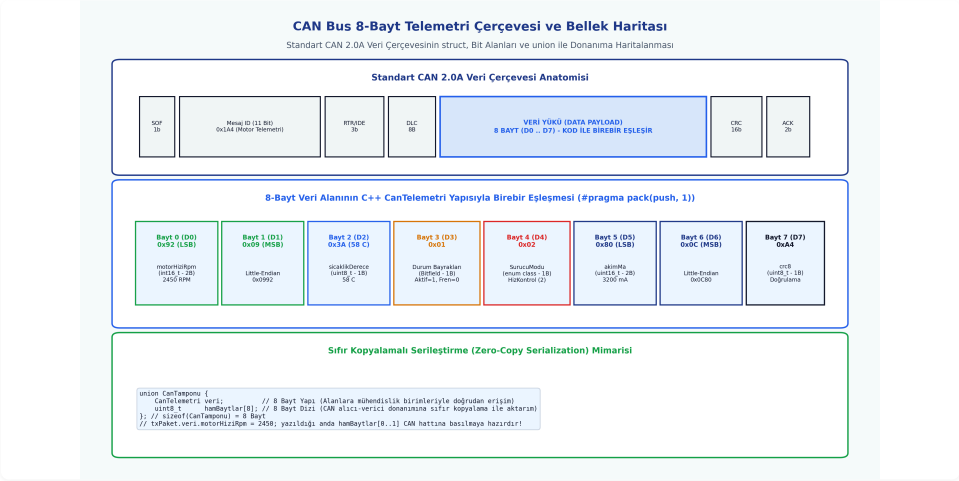
Karmaşık veri tiplerine ve fonksiyon işaretçilerine okunabilir takma adlar (*type aliases*) tanımlama:

```
5   typedef uint64_t Mikrosaniye_t;
6   typedef void (*KesmeIsleyici_old)(int);
7
8   // 2. Modern C++11 'using' bildirimi (Önerilen)
9   using Mikrosaniye   = uint64_t;
10  using CanMesajId    = uint16_t;
11  using KesmeIsleyici = void (*)(int);
12
13  int main() {
14      Mikrosaniye gecenSure = 250000ULL;
15      CanMesajId  motorId   = 0x1A4;
16      std::cout << "CAN ID: 0x" << std::hex << motorId << std::dec
17                << " | Sure: " << gecenSure << " us\n";
18      return 0;
19  }
```

```
1   CAN ID: 0x1a4 | Sure: 250000 us
```

# Mekatronik Vaka Analizi 1: CAN Bus Telemetri Çerçevesi

Standart 8-baytlık CAN 2.0A/B veri çerçevesinin `struct`, bit alanları, `enum class` ve `union` ile tasarımı:




- **8-Bayt Donanım Sınırı:** Standart CAN çerçevesi en fazla 8 bayt taşır; `#pragma pack(push, 1)` ile dolgu baytları sıfırlanarak 6 farklı telemetri verisi 8 bayta sığdırılır.
- **Sıfır Kopyalamalı Serileştirme:** `union CanTamponu` sayesinde yapıya yapılan atamalar anında `hamBaytlar[8]` dizisinden donanım hattına aktarılır.

# CAN Bus Çerçevesinin Veri Yapısı Tanımı

```
11          uint8_t trenDevrede : 1; // Bit 1
12          uint8_t asiriAkim   : 1; // Bit 2
13          uint8_t enkoderHata : 1; // Bit 3
14          uint8_t rezerve     : 4; // Bit 4-7
15      } bayraklar;                 // Bayt 3   (1 Bayt: 8 Bitlik durum alanı)
16      SurucuModu mod;              // Bayt 4   (1 Bayt: enum class)
17      uint16_t   akimMa;           // Bayt 5-6 (2 Bayt: 0 .. 65535 mA)
18      uint8_t    crc8;             // Bayt 7   (1 Bayt: Doğrulama özeti)
19  }; // Toplam Boyut: 2 + 1 + 1 + 1 + 2 + 1 = Tam 8 Bayt!
20  #pragma pack(pop)
21
22  union CanTamponu {
23      CanTelemetri veri;
24      uint8_t      hamBaytlar[8];
25  };
```

# CAN Bus Veri Paketleme ve Çözümleme Kodu

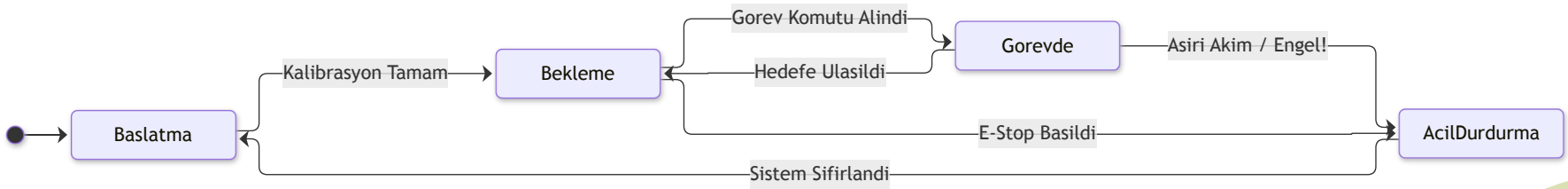
```
    txPaket.veri.sicaklikDerece      = 58;
    txPaket.veri.bayraklar.motorAktif = 1;
    txPaket.veri.mod                 = SurucuModu::HizKontrol;
    txPaket.veri.akimMa              = 3200;
    txPaket.veri.crc8                = 0xA4;

    // 2. CAN hattına giden 8 baytlık ham paketi yazdır
    std::cout << "CAN Cerceve Boyutu: " << sizeof(txPaket) << " Bayt\nCAN Hattina Giden: "
    for (int i = 0; i < 8; ++i) {
        std::cout << "0x" << std::hex << std::uppercase << std::setw(2)
                  << std::setfill('0') << static_cast<int>(txPaket.hamBaytlar[i]) << " ";
    }
    std::cout << std::dec << "\n";
    return 0;
}
```

```
CAN Cerceve Boyutu: 8 Bayt
CAN Hattina Giden: 0x92 0x09 0x3A 0x01 0x02 0x80 0x0C 0xA4
```

# Mekatronik Vaka Analizi 2: Robot Durum Makinesi (FSM)

Endüstriyel bir mobil robotun çalışma modları `enum class` ve `struct` birleşimiyle yönetilir:




- **Durum İzolasyonu:** `RobotDurum` numaralandırması geçersiz durum geçişlerini derleme zamanında önler.
- **Bağlam Yapısı (** `RobotBaglami` **):** Robotun anlık durumu, hızı ve hata bayrakları tek bir `struct` içinde taşınır.

# Robot Durum Makinesi C++ İmplementasyonu

```
24          case RobotDurum::Gorevde:
25              std::cout << "[FSM] Hareket suruyor. Hiz: " << robot.hedefHizMsn << " m/s\n";
26          case RobotDurum::AcilDurdurma:
27              std::cout << "[E-STOP] Engel algilandi! Motorlar kilitlendi. Hiz: 0 m/s\n"; br
28      }
29  }
30
31  int main() {
32      RobotBaglami agv = {RobotDurum::Baslatma, 0.0, false};
33      durumGuncelle(agv); // Baslatma -> Bekleme
34      durumGuncelle(agv); // Bekleme -> Gorevde
35      agv.engelAlgilandi = true;
36      durumGuncelle(agv); // AcilDurdurma!
37      return 0;
38  }
```

```
1  [FSM] Kalibrasyon tamamlandi -> BEKLEME moduna gecildi.
2  [FSM] Gorev alindi -> GOREVDE moduna gecildi.
3  [E-STOP] Engel algilandi! Motorlar kilitlendi. Hiz: 0 m/s
```

# Laboratuvar Alıştırması 1: IMU 6-Eksen Sensör Paketi

**Problem Tanımı:** Bir otonom insansız hava aracının (İHA) 6-eksenli IMU (*Inertial Measurement Unit*) sensöründen okunan 3-eksen ivmeölçer $(a_x, a_y, a_z)$ ve 3-eksen jiroskop $(g_x, g_y, g_z)$ verilerini modelleyen iç içe yapıyı ve sıfır-ofset kalibrasyon fonksiyonunu tasarlayınız.

- **Gereksinimler:**
  1. `Vektor3B` yapısı `double x, y, z;` üyelerini ve varsayılan kurucuyu içermelidir.
  2. `ImuPaketi` yapısı; paket numarası, `Vektor3B ivme` $(\mathrm{m/s^2})$ ve `Vektor3B jiroskop` $(\mathrm{deg/s})$ üyelerini barındırmalıdır.
  3. `kalibreEt(ImuPaketi& paket, const Vektor3B& jiroOfset)` fonksiyonu referans yoluyla jiroskop sapmasını düzeltmelidir.
  4. `raporla(const ImuPaketi& paket)` fonksiyonu veriyi kopyalamadan ekrana yazdırmalıdır.

> **IMU (Ataletsel Ölçüm Birimi):** İvmeölçer ve jiroskop sensörlerini birleştirerek aracın doğrusal ivmesini ve açısal hızını ölçen mekatronik algılayıcı.

# Alıştırma 1: IMU Kalibrasyon C++ Çözümü

```cpp
    p.jiroskop.x -= ofset.x;
    p.jiroskop.y -= ofset.y;
    p.jiroskop.z -= ofset.z;
}

int main() {
    ImuPaketi imu = {1, {0.02, -0.01, 9.81}, {1.25, -0.80, 0.45}};
    Vektor3B  sapmaOfseti(1.20, -0.80, 0.40);

    jiroskopKalibreEt(imu, sapmaOfseti);

    std::cout << "Kalibre Jiroskop (deg/s): X=" << imu.jiroskop.x
              << " Y=" << imu.jiroskop.y << " Z=" << imu.jiroskop.z << "\n";
    return 0;
}
```

```
Kalibre Jiroskop (deg/s): X=0.05 Y=0 Z=0.05
```

# Laboratuvar Alıştırması 2: 16-Bit ADC Donanım Kaydedicisi

**Problem Tanımı:** Bir mikrodenetleyicinin 16-bitlik (2 Bayt) ADC sonuç kaydedicisi şu bit yapısına sahiptir:

- **Bit 0–11 (12 Bit):** Ham ADC ölçüm değeri ($0 \ldots 4095$)
- **Bit 12–14 (3 Bit):** Kanal numarası ($0 \ldots 7$)
- **Bit 15 (1 Bit):** Dönüşüm tamamlandı bayrağı ( `hazir` : $0$ veya $1$)

Bu kaydediciyi hem `uint16_t hamVeri` hem de bit alanları olarak okuyabilen hibrit bir `union` ile modelleyiniz ve `0xB800` değerini çözümleyiniz.

> **Mühendislik İpucu:** 12-bitlik alanı tutabilmek için bit alanı üyesinin taban tipi en az 16-bitlik `uint16_t` olarak seçilmelidir ( `uint8_t` en fazla 8 bit tutabilir).

# Alıştırma 2: 16-Bit ADC Kaydedicisi C++ Çözümü

```cpp
};

int main() {
    AdcKaydedici16 adc;
    adc.alanlar.adcDegeri = 2048; // Yarı ölçek (1.65V)
    adc.alanlar.kanalNo   = 3;    // Kanal 3
    adc.alanlar.hazir     = 1;    // Ölçüm hazır

    std::cout << "Kaydedici Boyutu : " << sizeof(AdcKaydedici16) << " Bayt\n";
    std::cout << "16-Bit Hex Deger : 0x" << std::hex << std::uppercase << adc.hamKelime <<
    std::cout << "Kanal: " << adc.alanlar.kanalNo
              << " | ADC: " << adc.alanlar.adcDegeri
              << " | Gerilim: " << (adc.alanlar.adcDegeri * 3.3 / 4095.0) << " V\n";
    return 0;
}
```

```
Kaydedici Boyutu : 2 Bayt
16-Bit Hex Deger : 0xB800
Kanal: 3 | ADC: 2048 | Gerilim: 1.65 V
```

# Tasarım ve Uygulama Notları (Mühendislik İlkeleri)

Düşük seviyeli C++ veri yapılarında uyulması gereken yazılım mühendisliği kuralları:

- **Tasarım Notu 1 (Parametre Aktarımı):** Boyutu 8 baytı aşan tüm `struct` nesneleri fonksiyonlara değerle değil, `const Yapı&` referansı ile aktarılmalıdır.
- **Tasarım Notu 2 (Dahili Fragmantasyon):** Yapı üyelerini bellek boyutlarına göre **büyükten küçüğe** ( `double` $\rightarrow$ `int` $\rightarrow$ `short` $\rightarrow$ `char` ) sıralamak derleyici dolgusunu (*padding*) en aza indirir.
- **Uygulama Notu 1 ( `#pragma pack` Sınırı):** `#pragma pack(push, 1)` yalnızca donanım/ağ protokol çerçeveleri için kullanılmalı, sonrasında mutlaka `#pragma pack(pop)` ile normal hizalamaya dönülmelidir.
- **Uygulama Notu 2 ( `enum class` Tercihi):** Yeni C++ projelerinde isim çakışmalarını ve örtük dönüşüm hatalarını önlemek için klasik `enum` yerine daima `enum class` kullanılmalıdır.

# Hafta 6: Özet ve OOP Sınıflarına Köprü

Bu haftanın kazanımları, önümüzdeki hafta başlayacak Nesne Yönelimli Programlama sınıflarının temelini oluşturur:

- **Yapılardan Sınıflara Geçiş:** C++'ta `struct`, hem veri üyeleri hem de kurucu/yıkıcı metotlar barındırabilen tam yetenekli bir kullanıcı tanımlı türdür.
- **Kapsülleme İhtiyacı:** `struct` içindeki üyeler varsayılan olarak herkese açıktır (`public`). Veriyi dış müdahalelerden korumak ve geçerlilik denetimi yapmak için 7. haftada `class` ve `private` kapsülleme mimarisine geçeceğiz.
- **Donanım Seviyesi Hakimiyet:** `union`, bit alanları, Endian yönetimi ve `#pragma pack`, C++ dilini gömülü sistemlerde ve mekatronikte vazgeçilmez kılan düşük seviyeli bellek araçlarıdır.

# Gelecek Hafta: Hafta 7 Önizleme

Hafta 7: OOP'ye Giriş — Sınıflar, Nesneler ve Yaşam Döngüsü

- **Sınıf ( `class` ) ve Nesne ( `object` ) Paradigması:** Durum (*state*) ve davranış (*behavior*) bütünlüğü.
- **Erişim Belirleyicileri:** `public` , `private` ve `protected` görünürlük kuralları ve veri gizleme.
- **Nesne Yaşam Döngüsü:** Varsayılan, parametreli ve aşırı yüklenmiş kurucular (*constructors*) ile yıkıcılar (*destructors*).
- **Modern C++ Güvenceleri:** Üye ilklendirme listesi zorunlulukları ve `explicit` anahtar kelimesi ile örtük dönüşümlerin engellenmesi.
- **Arkadaş ( `friend` ) Mekanizması:** `friend` fonksiyonlar ve sınıflar ile kontrollü erişim.

# Kaynaklar ve İletişim

- Ders Deposu: github.com/marmara-mekatronik/MRM3049
- Kaynak Kitaplar:
  - Deitel & Deitel, *C++ How to Program* (Bölüm 22: Bits, Characters, C Strings and structs)
  - D.S. Malik, *Data Structures Using C++* (struct ve Soyut Veri Tipleri Bölümü)
- **İletişim:** Dr. Hüseyin Yüce (Marmara Üniversitesi Teknoloji Fakültesi / Mekatronik Mühendisliği)

Nesne Yönelimli Programlama • Hafta 6: Yapılar, Birlikler ve Numaralandırmalar